

Практическое занятие 3. Особые точки и стереовидение

Особые точки

Особые точки, выделенные из объектов, обладают такими свойствами, как инвариантность относительно смещения, поворот, изменение масштаба, изменение яркости. Эти свойства позволяют использовать особые точки при классификации объектов.

Одним из наиболее популярных алгоритмов, включающих дескриптор и детектор характерных точек изображения, является SIFT (Scale Invariant Feature Transform), предложенный D.G. Lowe в 1999 году.

Данный алгоритм представляет собой локальную гистограмму направлений градиентов изображения.

Принцип работы алгоритма SIFT заключается в вычислении свертки исходного изображения с ядром Гаусса при изменяющемся параметре сглаживания. После этого происходит преобразование изображений к одному размеру, и вычисляется их разность.

Далее выполняется сравнение каждого пикселя на изображении с восемью соседними пикселями при тех же параметрах и масштабе, с девятью соседними пикселями в большем масштабе и с девятью в меньшем масштабе. Пиксели, в которых локальные экстремумы превосходят заданный порог, выбираются как характерные точки.

Для каждой выбранной точки вычисляется определенный локальный дескриптор, который характеризует направление градиентов в данной окрестности пикселей.

В методе SIFT (Scale Invariant Feature Transform) детектирование производится с построения гауссова масштабируемого пространства, состоящего из пирамиды гауссианов и пирамиды разностей гауссианов. Функции для гауссианов и их разностей соответственно имеют следующий вид:

$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$ – гауссиан изображения

$D(x, y, \sigma) = (G(x, y, k\sigma) - G(x, y, \sigma)) * I(x, y) = L(x, y, k\sigma) -$

$L(x, y, \sigma)$ – разность гауссианов

Гауссово пространство инвариантно относительно большинства видов преобразования изображений: масштабирование, поворот, смещение, изменение яркости и т. д.

Октава (пирамида) в SIFT (Scale-Invariant Feature Transform) - это набор изображений, полученных путем многократного сглаживания и уменьшения размера исходного изображения. Каждая октава включает в себя несколько масштабов, и каждый масштаб содержит образцы особенностей изображения. Это позволяет обнаруживать особенности на изображениях разного масштаба и поворота и делает алгоритм SIFT масштабно инвариантным.

Пакет OpenCV содержит функцию `cv2.xfeatures2d.SIFT_create()`, которая позволяет использовать алгоритм SIFT для обнаружения особенностей на изображениях.

Метод ORB предложен в 2011г. В его основе лежит комбинация таких алгоритмов как детектор FAST (**F**eatures from **A**ccelerated **S**egment **T**est) и дескриптор BRIEF (**B**inary **R**obust **I**ndependent **E**lementary **F**eatures) с некоторыми улучшениями.

Функция в OpenCV:

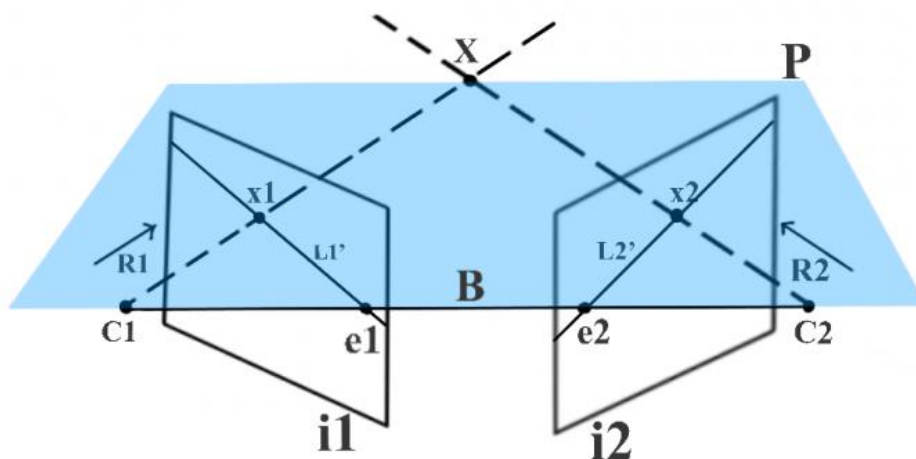
`orb = cv2.ORB_create(, nfeatures, scaleFactor, nlevels, edgeThreshold, firstLevel, WTA_K, scoreType, patchSize, fastThreshold)`

- `nfeatures` : Максимальное количество сохраняемых функций, по умолчанию - 500.
- `scaleFactor`: Коэффициент извлечения пирамиды, больше 1, по умолчанию 1,2. `scaleFactor == 2` представляет собой классическую пирамиду, в каждом слое в 4 раза меньше пикселей, чем в предыдущем слое, но такой большой масштабный коэффициент значительно снизит оценку соответствия характеристик. С другой стороны, коэффициент масштабирования, который слишком близок к 1, будет означать, что он покрывает определенный диапазон масштабирования, вам потребуется больше уровней пирамиды, поэтому это повлияет на скорость.
- `nlevels` : Количество уровней пирамиды, по умолчанию 8. Линейный размер минимального уровня будет равен $\text{input_image_linear_size} / \text{pow}(\text{scaleFactor}, \text{nlevels} - \text{firstLevel})$.
- `edgeThreshold` : Порог границы, который представляет собой размер границы не обнаруженных объектов, по умолчанию 31. Он должен примерно соответствовать параметру `patchSize`.
- `firstLevel` : Уровень пирамиды, на котором размещается исходное изображение, по умолчанию - 0. Все предыдущие слои представляют собой увеличенные исходные изображения.
- `WTA_K` : Количество баллов для каждого элемента объектно-ориентированного дескриптора BRIEF. Значение по умолчанию 2 означает, что нужно взять случайную пару точек и сравнить их яркость, чтобы получить ответ 0/1. Другие возможные значения - 3 и 4. Например, это означает, что нам нужны 3 случайные точки (конечно, координаты этих точек случайны, но они генерируют предопределенное начальное число, поэтому краткий дескриптор каждого элемента вычисляет детерминированный прямоугольник пикселей), максимальная яркость найденного победителя и Индекс вывода (0, 1 или 2). Такой вывод будет занимать 2 бита, поэтому для него требуется специальный вариант расстояния Хэмминга, обозначенный как `NORM_HAMMING2` (2 бита / бит). Когда `WTA_K =`

- 4, мы берем 4 случайные точки для вычисления каждого бина (он также будет занимать 2 бита, которые могут быть 0, 1, 2 или 3).
- `scoreType` : Значение по умолчанию `HARRIS_SCORE` указывает, что алгоритм Харриса используется для сортировки функций (оценка записывается как `KeyPoint :: score`, которая используется для сохранения лучших функций); `FAST_SCORE` - это замещающее значение параметра, значение ключа, генерируемое параметром, немного нестабильно, но вычисляется быстрее.
 - `patchSize` : Размер матрицы, используемой объектно-ориентированным дескриптором BRIEF. Значение по умолчанию - 31. Конечно, на меньшем уровне пирамиды воспринимаемая область изображения, покрываемая элементом, будет больше.
 - `fastThreshold` : Порог детектора функции FAST, по умолчанию 20

Стереовидение.

Стереопсис (от греческого *στερεο-* стерео-, что означает «твердый», и *ὄψις* *opsis*, «внешний вид, взгляд») - это термин, который чаще всего используется для обозначения восприятия глубины и полученной трехмерной структуры на основе визуальной информации, полученной от двух глаз людей с нормально развитым бинокулярным зрением (Википедия)



$C1$ и $C2$ — известные трехмерные положения левой и правой камер соответственно.

$x1$ — это изображение трехмерной точки X , снятое левой камерой, а $x2$ — изображение X , снятое правой камерой. $x1$ и $x2$ называются соответствующими точками, потому что они являются проекцией одной и той же трехмерной точки. Мы используем $x1$ и $C1$, чтобы найти $L1$, и $x2$ и $C2$,

чтобы найти $L2$. Следовательно, мы можем использовать триангуляцию для нахождения X .

3D-точка X снимается в точках $x1$ и $x2$ камерами в $C1$ и $C2$ соответственно.

Поскольку $x1$ является проекцией X , если мы попытаемся удлинить луч $R1$ из $C1$, который проходит через $x1$, он также должен пройти через X . Этот луч $R1$ фиксируется как линия $L2$, а X фиксируется как $x2$ на изображении $i2$.

Поскольку X лежит на $R1$, $x2$ должен лежать на $L2$. Таким образом, возможное местоположение $x2$ ограничено одной строкой, и, следовательно, мы можем сказать, что пространство поиска для пикселя в изображении $i2$, соответствующем пикселю $x1$, сокращается до одной строки $L2$. Мы используем эпиполярную геометрию, чтобы найти $L2$.

На рисунке $e2$ — это проекция центра камеры $C1$ на изображении $i2$, а $e1$ — это проекция центра камеры $C2$ на изображении $i1$. Технический термин для $e1$ и $e2$ — эпиполь.

Следовательно, в настройке геометрии с двумя ракурсами эпиполь — это изображение центра камеры одного ракурса в другом ракурсе. Линия, соединяющая центры двух камер, называется базовой линией.

Следовательно, эпиполь также можно определить как пересечение базовой линии с плоскостью изображения.

С помощью $R1$ и базовой линии мы можем определить плоскость P . Эта плоскость также содержит X , $C1$, $x1$, $x2$ и $C2$. Мы называем эту плоскость эпиполярной плоскостью.

Линия, полученная от пересечения эпиполярной плоскости и плоскости изображения, называется эпиполярной линией.

Следовательно, в нашем примере $L2$ — эпиполярная линия. Для разных значений X у нас будут разные эпиполярные плоскости и, следовательно, разные эпиполярные линии.

Все эпиполярные плоскости пересекаются на базовой линии, и все эпиполярные линии пересекаются в эпиполе.

Все это вместе образует эпиполярную геометрию.

(см. более подробно в лекциях)

Задание.

1. Определение особых точек на изображении с помощью детекторов SIFT и ORB.

- 1) С помощью ниже описанной программы 1 с детектором SIFT найдите на изображении особые точки. Используйте те же два типа изображений, что и в предыдущих работах.
- 2) С помощью ниже описанной программы 2 с детектором ORB найдите на изображении особые точки. Используйте те же два типа изображений.
- 3) Сравните результаты, сделайте выводы.

Программа 1.

```
import cv2
# Загрузка изображения
img = cv2.imread('image.jpg')
# Создание объекта детектора SIFT
sift = cv2.xfeatures2d.SIFT_create()
# Нахождение особых точек и описаний признаков на изображении
keypoints, descriptors = sift.detectAndCompute(img, None)
# Отмечаем найденные особые точки на исходном изображении
img_keypoints = cv2.drawKeypoints(img, keypoints, None)
# Выводим изображение с отмеченными особыми точками
cv2.imshow("Image with keypoints", img_keypoints)
# Вывод количества найденных особых точек
print("Found %d keypoints" % len(keypoints))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Программа 2.

```
import cv2
# Загрузка изображения
img = cv2.imread('ki.png')
# Создание объекта детектора ORB
orb = cv2.ORB_create()
# Нахождение особых точек и описаний признаков на изображении
keypoints, descriptors = orb.detectAndCompute(img, None)
# Отмечаем найденные особые точки на исходном изображении
img_keypoints = cv2.drawKeypoints(img, keypoints, None)
# Выводим изображение с отмеченными особыми точками
cv2.imshow("Image with keypoints", img_keypoints)
# Вывод количества найденных особых точек
print("Found %d keypoints" % len(keypoints))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

2. Стереовидение. Определение диспаратности.

Подготовить два фото одного и того же объекта, снятого из двух разных точек (расстояние между ними 10-15 см).

Довести до рабочего состояния программу, приведенную ниже.

Провести эксперимент по обработке выбранных двух изображений.

```
# Чтение левого и правого изображений.
imgL = cv2.imread("../im0.png",0)
imgR = cv2.imread("../im1.png",0)
# Настройка параметров для алгоритма StereoSGBM
minDisparity = 0;
numDisparities = 64;
blockSize = 8;
disp12MaxDiff = 1;
uniquenessRatio = 10;
speckleWindowSize = 10;
speckleRange = 8;
# Создание объекта алгоритма StereoSGBM
stereo = cv2.StereoSGBM_create(minDisparity = minDisparity,
                               numDisparities = numDisparities,
                               blockSize = blockSize,
                               disp12MaxDiff = disp12MaxDiff,
                               uniquenessRatio = uniquenessRatio,
                               speckleWindowSize = speckleWindowSize,
                               speckleRange = speckleRange
)
# Вычисление диспаритета с использованием алгоритма StereoSGBM
disp = stereo.compute(imgL, imgR).astype(np.float32)
disp = cv2.normalize(disp,0,255,cv2.NORM_MINMAX)
# Отображение карты диспаратности
cv2.imshow("disparity",disp)
cv2.waitKey(0)
```

Источник:

<https://waksoft.susu.ru/2021/01/26/vvedenie-v-epipolyarnuyu-geometriyu-i-stereozrenie/>